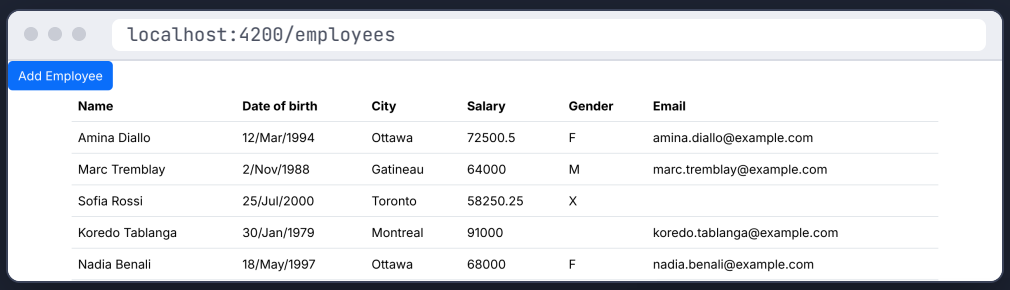
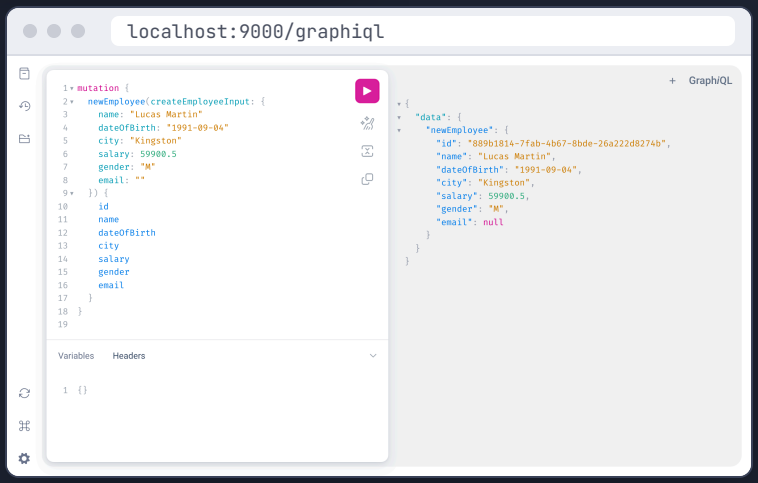
SEG3502 · ARCHITECTURE ET CONCEPTION LOGICIELLE

Laboratoire 8
Services Web GraphQL

Exercice évalué : compléter le serveur GraphQL (Spring Boot, Kotlin, MongoDB) pour que le client Angular des employés fonctionne. La liste vient de la requête **employees**, le formulaire enregistre un employé avec la mutation **newEmployee**.





ÉQUIPE

Brayan Adou — 300433616
Écriture : mutation newEmployee, validation, tests, README, captures et rapport

Hjiyej Andaloussi Elghali — 300379897
Lecture : dépôt, requête employees, page GraphiQL, tests et données d'exemple

COURS
SEG3502 A00
Université d'Ottawa

REMISE
11 novembre 2026

DÉPÔT GITHUB
Lien remis sur Brightspace
avec ce rapport

01 Description du travail réalisé

Le serveur GraphQL des employés est complété : schéma `employee.graphqls` et résolveurs dans `EmployeesResolver`, sans modifier le client Angular fourni.

Le code de départ contient le client de saisie d'employés du laboratoire 5, déjà relié par Apollo à `http://localhost:9000/graphql`, et un serveur Spring Boot avec l'entité `Employee`, `EmployeesRepository` (MongoDB), `CreateEmployeeInput` et la configuration CORS et sécurité. Le schéma était vide et le résolveur ne contenait aucune méthode. Comme le demande l'énoncé, les noms, arguments et champs à implémenter ont été relevés dans le client (`client/src/app/service/employee.service.ts`).

Ce que le client envoie au serveur

OPÉRATION DU CLIENT	TYPE	CHAMP	ARGUMENTS	CHAMPS DEMANDÉS
`GET_EMPLOYEES` watchQuery au démarrage	Query	`employees`	aucun	id, name, dateOfBirth, city, salary, gender, email
`ADD_EMPLOYEE` envoi du formulaire	Mutation	`newEmployee`	`createEmployeeInput` : name, dateOfBirth, city (String), salary (Float), gender, email (String)	

Choix faits dans le serveur

- **Mêmes noms que le client** : `employees` , `newEmployee` , `createEmployeeInput` , types `Employee` et `CreateEmployeeInput` . `gender` et `email` peuvent être null, car ils sont facultatifs dans le formulaire.
- **Champs de l'input facultatifs**, comme dans `CreateEmployeeInput.kt` et le `CreateBookInput` de l'exemple du PDF : le résolveur vérifie lui même les champs obligatoires.
- **Id créé par le serveur** avec `UUID.randomUUID()` , comme `newBook` dans l'exemple. L'id vaut `""` par défaut dans `Employee` : sans cela, chaque ajout remplacerait le précédent dans MongoDB.
- **Mêmes règles que le formulaire** : nom, date de naissance, ville et salaire obligatoires, date au format `yyyy-MM-dd` (envoyé par `<input type="date">` et lu par le `DatePipe` de la liste), genre M, F ou X, courriel valide. Une entrée invalide envoyée depuis GraphiQL donne une erreur `BAD_REQUEST` claire (`@GraphQlExceptionHandler`) et rien n'est enregistré.
- **Champs vides** : le client envoie `""` pour un genre ou un courriel non rempli ; le serveur les enregistre comme `null` et retire les espaces autour des textes.

Circulation des données

1. Client Angular
`EmployeeService` (Apollo) envoie `GET_EMPLOYEES` au démarrage et `ADD_EMPLOYEE` au clic sur Submit, en POST sur `/graphql`.

→

2. Spring for GraphQL
Valide la requête avec `employee.graphqls`, puis appelle `employees()` (`@QueryMapping`) ou `newEmployee()` (`@MutationMapping`).

→

3. MongoDB
`EmployeesRepository` lit ou enregistre les documents de la collection `employee`, base `books-mongo`, port 27017.

Le client ajoute ensuite l'employé renvoyé par `newEmployee` à sa liste et revient sur `/employees` .

01 Schéma et résolveurs

Les deux fichiers demandés par l'exercice, dans le dossier `server/src/main`.

`resources/graphql/employee.graphqls`

```
type Query {
    employees: [Employee]
}

type Employee {
    id: ID!
    name: String!
    dateOfBirth: String!
    city: String!
    salary: Float!
    gender: String
    email: String
}

input CreateEmployeeInput {
    name: String
    dateOfBirth: String
    city: String
    salary: Float
    gender: String
    email: String
}

type Mutation {
    newEmployee(createEmployeeInput:
        CreateEmployeeInput!): Employee!
}
```

`kotlin/seg3x02/employeeGql/resolvers/EmployeesResolver.kt`

```
@Controller
class EmployeesResolver(
        private val employeesRepository: EmployeesRepository) {

    @QueryMapping
    fun employees(): List<Employee> {
        return employeesRepository.findAll()
    }

    @MutationMapping
    fun newEmployee(@Argument("createEmployeeInput")
                    input: CreateEmployeeInput): Employee {
        val employee = Employee(
            requiredText(input.name, "name"),
            requiredDate(input.dateOfBirth),
            requiredText(input.city, "city"),
            input.salary ?: throw InvalidEmployeeInputException(
                "salary is required."),
            optionalGender(input.gender),
            optionalEmail(input.email))
        employee.id = UUID.randomUUID().toString()
        employeesRepository.save(employee)
        return employee
    }

    // Entrée invalide : erreur BAD_REQUEST avec un message clair
    @GraphQlExceptionHandler
    fun handleInvalidInput(ex: InvalidEmployeeInputException,
                           env: DataFetchingEnvironment): GraphQLError =
        GraphqlErrorBuilder.newError(env)
            .errorType(ErrorType.BAD_REQUEST)
            .message("Invalid employee: " + ex.message).build()

    // requiredText, requiredDate, optionalGender, optionalEmail :
    // champs obligatoires, LocalDate.parse, M/F/X, "" devient null
}
```

Trois problèmes trouvés et corrigés pendant l'intégration

1. **Un seul employé gardé dans MongoDB.** `Employee.id` vaut `""` par défaut : tous les ajouts avaient le même `_id`. Le résolveur crée maintenant un UUID (vérifié par un test).
2. **`npm install` s'arrêtait (ERESOLVE).** `apollo-angular` 5.0.2, version du code de départ, déclare Angular 14 à 16, alors que le client utilise Angular 18. `client/.npmrc` active `legacy-peer-deps` ; le code du client n'est pas modifié.
3. **GraphiQL restait sur « Loading… ».** La page de Spring for GraphQL 1.3.1 charge `graphiql` sans version depuis unpkg ; depuis GraphiQL 4 (mai 2025), `graphiql.min.js` n'existe plus. Une copie de la page dans `src/main/resources/graphiql/` charge GraphiQL 3.9.0 (vérifié par un test).

02 Répartition, tests et commits

Partage par opération : lecture (`employees`) et écriture (`newEmployee`), chacune avec sa partie du schéma, son résolveur et ses tests.

Hijiyej Andaloussi Elghali — LECTURE · COMMIT 1

- Création du dépôt : code de départ, `.gitignore`, `.gitattributes`, `client/.npmrc`
- Schéma : `Query.employees` et type `Employee`
- `EmployeesResolver.employees()` (`@QueryMapping`)
- Page GraphiQL avec versions fixées
- 6 tests : `EmployeesQueryTests`, `GraphiQlPageTests`
- `scripts/seed-employees.sh` : 4 employés d'exemple

Brayan Adou — ÉCRITURE · COMMIT 2

- Schéma : `input CreateEmployeeInput` et `Mutation.newEmployee`
- `EmployeesResolver.newEmployee()` (`@MutationMapping`), id UUID
- Validation et erreurs `BAD_REQUEST` (`@GraphQlExceptionHandler`)
- 10 tests : `EmployeesMutationTests`
- README, captures d'écran et rapport (`docs/`)

Tests du serveur (JUnit 5, Spring for GraphQL)

Les tests passent par le vrai schéma et le vrai résolveur (`@GraphQlTest`, `GraphQlTester`), avec les requêtes copiées du client dans `src/test/resources/graphql-test/`. MongoDB est simulé (`@MockBean`) : `./gradlew test` ne demande aucun serveur MongoDB.

FICHIER DE TEST	CE QUI EST VÉRIFIÉ	DÉPART	LECTURE	ÉCRITURE
`EmployeeGqlApplicationTests`	Le contexte Spring démarre	1		
`EmployeesQueryTests`	Requête du client : liste vide, les 7 champs, gender et email null, types du schéma, champ inconnu refusé		5	
`GraphiQlPageTests`	`/graphiql` charge GraphiQL 3.9.0 et non plus la version sans numéro		1	
`EmployeesMutationTests`	Mutation du client : employé enregistré et renvoyé, ids UUID différents, "" devenu null, 5 valeurs refusées (BAD_REQUEST, rien d'enregistré), salaire absent, ajout puis relecture			10
Total : 17		**1**	**6**	**10**

Trois défauts introduits volontairement dans le résolveur (id non créé, mauvais type d'erreur, "" gardé) ont fait échouer 9 tests, puis le code a été rétabli. Les deux opérations du client ont aussi été validées contre `employee.graphqls` avec la bibliothèque `graphql` du client : aucune erreur.

Deux commits sur le même dépôt

Commit 1 · Hijiyej Andaloussi Elghali

Crée le dépôt avec le code de départ et la partie lecture. Le serveur démarre et la liste des employés s'affiche dans le client (vide ou avec les données d'exemple). **7 tests réussissent.**

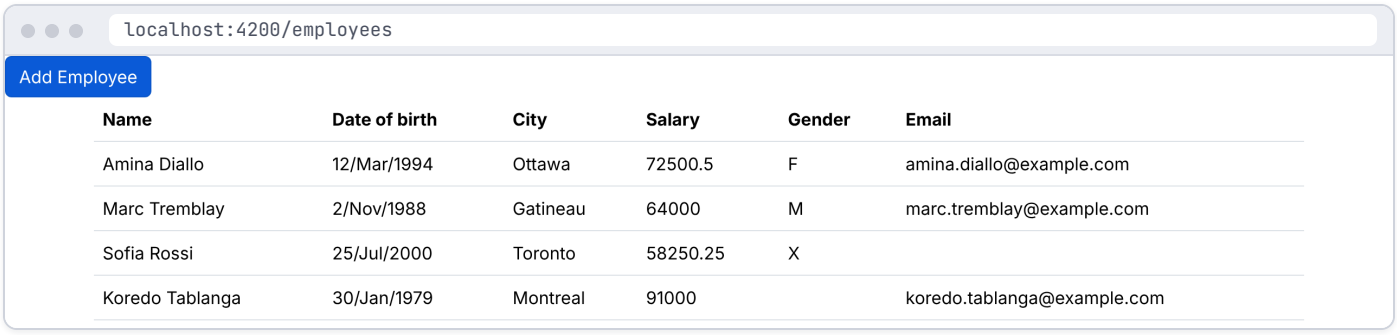
Commit 2 · Brayan Adou

Récupère le dépôt (`git pull`), remplace `employee.graphqls`, `EmployeesResolver.kt` et le README, ajoute les tests de la mutation et `docs/`. Le formulaire Add Employee fonctionne. **17 tests réussissent.**

Vérifié avant la remise. Les deux commits ont été rejoués dans un dépôt Git de test. Après chaque commit, le dépôt a été cloné à neuf, puis `./gradlew test` (serveur), `npm install` et `ng build` (client) ont réussi. Les scripts `gradlew` et `seed-employees.sh` restent exécutables.

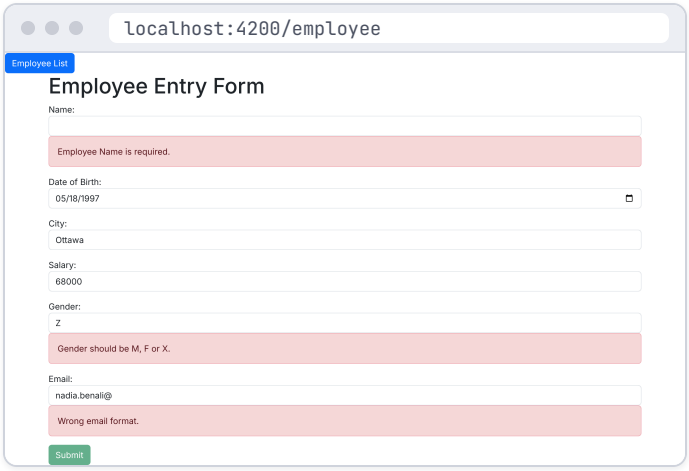
03 Captures d'écran

Serveur lancé avec `./gradlew bootRun`, MongoDB sur le port 27017, 4 employés ajoutés avec `scripts/seed-employees.sh`, client lancé avec `npm start` et ouvert à `http://localhost:4200/`.



Name	Date of birth	City	Salary	Gender	Email
Amina Diallo	12/Mar/1994	Ottawa	72500.5	F	amina.diallo@example.com
Marc Tremblay	2/Nov/1988	Gatineau	64000	M	marc.tremblay@example.com
Sofia Rossi	25/Jul/2000	Toronto	58250.25	X	
Koredo Tablanga	30/Jan/1979	Montreal	91000		koredo.tablanga@example.com

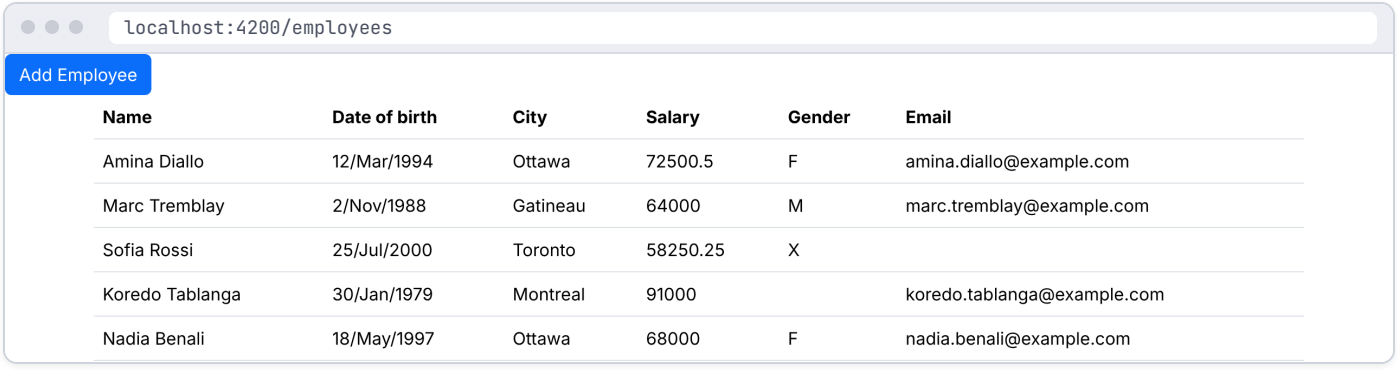
Liste des employés (requête employees). Au démarrage, le client envoie `GET_EMPLOYEES` ; les 4 employés viennent de MongoDB. Les cases vides de Gender et Email sont des valeurs null.



Validation du formulaire. Nom vide, genre Z et courriel incomplet : messages sous les champs, bouton Submit désactivé, aucune requête envoyée.



Formulaire valide, juste avant le clic sur Submit.

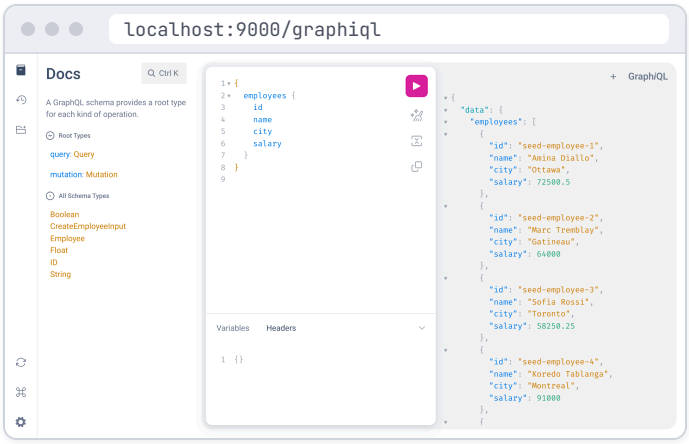


Name	Date of birth	City	Salary	Gender	Email
Amina Diallo	12/Mar/1994	Ottawa	72500.5	F	amina.diallo@example.com
Marc Tremblay	2/Nov/1988	Gatineau	64000	M	marc.tremblay@example.com
Sofia Rossi	25/Jul/2000	Toronto	58250.25	X	
Koredo Tablanga	30/Jan/1979	Montreal	91000		koredo.tablanga@example.com
Nadia Benali	18/May/1997	Ottawa	68000	F	nadia.benali@example.com

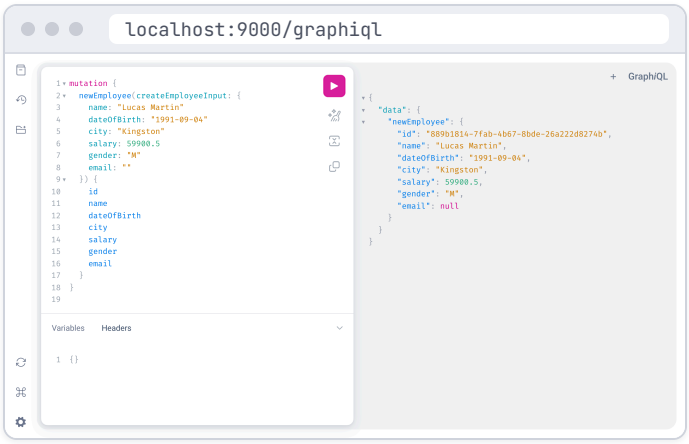
Après Submit (mutation newEmployee). Le serveur enregistre Nadia Benali et renvoie l'employé avec son id ; le client revient à la liste. Après un rechargement de la page, la ligne est toujours là : elle vient bien de MongoDB.

03 Captures d'écran (suite)

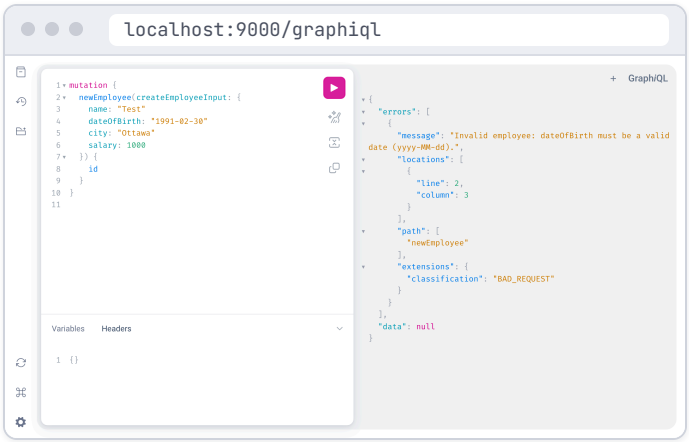
GraphiQL, ouvert à `http://localhost:9000/graphiql`, envoie les mêmes requêtes que le client.



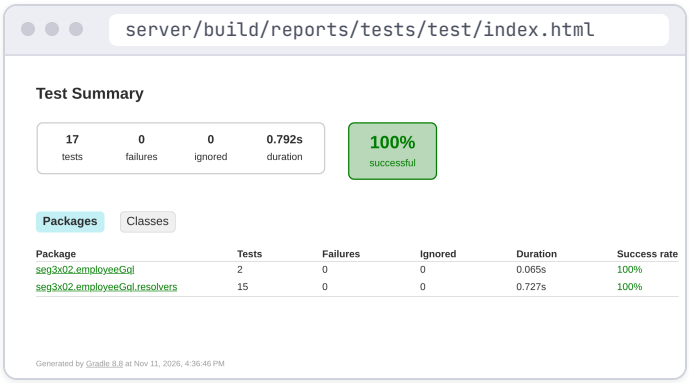
Requête employees. Le panneau Docs montre les types du schéma : Query, Mutation, Employee et CreateEmployeeInput.



Mutation newEmployee. L'id UUID est créé par le serveur ; le courriel vide est enregistré comme null.



Entrée invalide. Date du 30 février : erreur BAD_REQUEST avec un message clair, data null, rien n'est enregistré.



Tests du serveur. `./gradlew test` : 17 tests, 0 échec.

Ce que montrent les captures

- Le client fourni fonctionne avec le serveur complété : la liste vient de la requête `employees`, le formulaire ajoute un employé avec la mutation `newEmployee`.
- Les données sont gardées dans MongoDB : elles restent après un rechargement de la page.
- Le serveur refuse une entrée invalide avec un message clair, et 17 tests automatiques réussissent.

04 Installer et exécuter le projet

Deux dossiers dans le dépôt : `server/` (Spring Boot 3.3.1, Kotlin, Gradle 8.8) et `client/` (Angular 18, Apollo). Trois terminaux sont utiles : MongoDB et données, serveur, client.

Prérequis

- **Docker** pour MongoDB.
- **Java 21** : le projet utilise une toolchain Java 21 ; vérifier avec `java -version`.
- **Node.js** 18.19+, 20.11+ ou 22+ (`node -v`) ; npm est fourni avec Node.js.
- **Git** (Git Bash sous Windows pour le script de données).

Étapes

1 Cloner le dépôt

```
git clone <lien du dépôt remis sur Brightspace>
cd <dossier du dépôt>
```

2 Démarrer MongoDB (port 27017)

```
docker run -d -p 27017:27017 --name mongodb mongo
```

Les fois suivantes : `docker start mongodb`. Le serveur utilise la base `books-mongo`.

3 Démarrer le serveur GraphQL (port 9000)

```
cd server
./gradlew bootRun                # Windows : gradlew.bat bootRun
```

Point de terminaison : `http://localhost:9000/graphql` · GraphiQL : `http://localhost:9000/graphiql`

4 Ajouter les données d'exemple (facultatif, à la racine du dépôt)

```
bash scripts/seed-employees.sh
```

Ajoute 4 employés par le conteneur Docker `mongodb` (ou par `mongosh` s'il est installé). Le script peut être relancé : il remplace ses 4 employés sans toucher aux autres.

5 Démarrer le client Angular (port 4200)

```
cd client
npm install
npm start                        # ou : npx ng serve
```

6 Utiliser l'application

Ouvrir `http://localhost:4200/` : le formulaire s'affiche. **Employee List** affiche la liste, **Add Employee** revient au formulaire ; après **Submit**, le nouvel employé apparaît dans la liste. Arrêter avec `Ctrl + C` dans chaque terminal, puis `docker stop mongodb`.

Tests

```
cd server
./gradlew test                   # 17 tests, MongoDB n'est pas nécessaire
```

> Ouvrir le client avec `localhost`, pas `127.0.0.1` : le serveur n'accepte que l'origine `http://localhost:4200` (CORS dans `WebConfiguration.kt`). Sinon, la liste reste vide et le formulaire ne fait rien.